

Comparación: app ToDo vanilla JS → React

Lo que pidió Leo: «que compares lo que tenés en un branch con otro, para que te des cuenta que cambia el código». Este documento es esa comparación, lista para explicar en clase · analogía de la cocina: misma receta, misma cocina, cambió el CUADERNO donde se anota (DOM a mano → componentes)

PARTE 1

Los dos branches

Branch	Qué contiene	Cómo se ve
<code>feature/clase13-dto-comentarios</code> commit 4c606a9	La app aprobada: <code>backend/</code> (Express) + <code>frontend/</code> (HTML/CSS/JS vanilla)	Servidor estático o doble clic en <code>index.html</code>
<code>feature/react</code> commit 5388bdc	El mismo backend + <code>frontend-react/</code> (Vite + React 19 + TS + Router 7)	<code>npm run dev</code> → <code>http://localhost:5173</code>

👉 **Regla de oro:** el backend NO se tocó. Misma API, misma base, mismos estilos. Solo cambió la capa de la vista — que es exactamente lo que Leo quería que veas.

Cómo correr cada versión

```
Vanilla (branch feature/clase13-dto-comentarios)
cd backend && npm start           # API en :3000 — igual en las dos
frontend/index.html             # con servidor estático

React (branch feature/react)
cd backend && npm start           # API en :3000 (idéntico)
cd frontend-react && npm run dev # app en :5173 — el proxy de Vite
                                # manda /api → :3000 (sin CORS, sin URLs absolutas)
```

PARTE 2

El antes y el después, archivo por archivo

Vanilla (frontend/)	React (frontend- react/src/)	Qué pasó
<code>index.html</code> + <code>area.html</code> 2 páginas, ?id= en la URL	<code>main.tsx</code> + <code>App.tsx</code> 1 página, 3 rutas: <code>/</code> <code>/areas/:id</code> <code>*</code>	Las páginas ya no son archivos: son rutas (SPA — el <code><div id="root"></code> vacío que mostró Leo)
<code>js/model.js</code> (117 líneas)	<code>api.ts</code>	Casi no cambió: mismas funciones, verbos y errores. Solo URL relativa (proxy) + ahora devuelve tipos
<code>js/view.js</code> (158 líneas)	<code>componentes/</code> (TarjetaTarea, ChipEstatus, TarjetaComentario, Iconos)	<code>pintarLista/pintarDetalle/pintarComentarios</code> (innerHTML con template literals) → componentes: funciones que retornan JSX
<code>js/controller.js</code> (192 líneas)	<code>paginas/PaginaLista.tsx</code>	El director de eventos (addEventListener, editandold, getElementById) → estado + hooks y el HTML de la página vive en el componente
<code>js/detalle.js</code> (137 líneas)	<code>paginas/PaginaDetalle.tsx</code>	Igual: <code>URLSearchParams</code> → <code>useParams()</code> ; <code>ocultarSecciones()</code> → render condicional
—	<code>tipos.ts</code>	Nuevo: TypeScript define la forma de Tarea/Comentario (el mismo contrato del backend, del lado del front)
—	<code>PaginaNoEncontrada.tsx</code>	Nuevo: la ruta <code>*</code> — en vanilla no había 404 en el front

Lo que DESAPARECIÓ (el punto de Leo)

Herramienta vanilla	Reemplazo en React	Por qué
<code>document.getElementById</code> / <code>querySelector</code>	<code>useState</code> + <code>value=</code> <code>{...}</code>	No se agarran cajitas: cada input es controlado por estado
<code>innerHTML = '...'</code>	<code>JSX ({tarea.nombre})</code>	El HTML se escribe en el componente, no como string
<code>escapeHtml()</code> (26 líneas manuales)	Nada — React escapa solo	La protección XSS que arreglamos en la auditoría viene gratis
<code>addEventListener</code> + delegación (<code>closest('button[data-accion]')</code>)	<code>onSubmit</code> , <code>onClick</code> , <code>onChange</code>	El evento se declara donde está el botón
<code>classList.toggle('hidden')</code>	Render condicional (<code>{tarea && ...}</code>)	Si no hay tarea, la sección no se dibuja
<code>form.reset()</code> + <code>volcarEnFormulario()</code>	Estado inicial + <code>key</code>	Al cambiar la key React recrea el componente
<code>btn.disabled = true</code> (anti doble-click)	<code>enviando</code> (estado)	El botón se deshabilita mientras el estado diga "enviando"
El bug del aviso "No hay tareas" (innerHTML="" lo borraba)	No puede existir	Sin innerHTML: el aviso es render condicional
<code>tarea.html?id=13</code> (recarga la página)	<code><Link</code> <code>to="/tareas/13"></code>	Navega SIN recargar (SPA)
<code>URLSearchParams(location.search)</code>	<code>useParams()</code>	La ruta ya trae el <code>:id</code>

PARTE 4

Lo que quedó IGUAL

- `backend/` **completo** — ni una línea tocada (API, DB, DTO, PATCH, comentarios).
- `styles.css` — copiado tal cual (440 líneas); solo `class` → `className` .
- **El model** (`api.ts`) — las mismas 8 funciones de `model.js` .
- **El comportamiento:** filtros con default "Pendiente", PATCH por chip, autor en localStorage (misma key `comentario-autor`), 404 oculta secciones, errores del backend en pantalla.
- **La base** (`tareas.db`) — ni una migración.

PARTE 5

Verificación y stack

Verificado el 20/9

✓ `npm run build` → TypeScript + Vite, **0 errores**.

✓ `npm run lint` → **0 warnings, 0 errores**.

✓ E2E contra la API (8/8): crear → PATCH estatus → GET detalle → 2 comentarios → borrar 1 → filtro → DELETE → limpieza.

✓ Render con Chrome headless: lista con datos reales (proxy OK) · detalle con 4 comentarios · tarea inexistente → "Tarea no encontrada" + secciones ocultas · ruta rara → "Ruta no encontrada".

Stack instalado (el mismo de Iris)

```
React 19.3 + React Router 7.18 + TypeScript 6 + Vite 8  
npm create vite@latest · react-router-dom · proxy /api → :3000 en vite.config.ts
```

Apoyos: transcript clase 14 (`_clases/17-09-26.txt`) · TAREAS.md · COMPARACION_VANILLA_REACT.md (el mismo contenido en markdown, en el repo)